

```

1. #include <iostream>
2. #include <fstream>
3. #include <algorithm>
4. #include <math.h>
5. #include <vector>
6. #include <queue>
7. #include <stack>
8.
9. using namespace std;
10.
11. ifstream fin("citire.in");
12. ofstream fout("afisare.out");
13.
14.
15. class Graph
16. {
17. public:
18.     struct Edge
19.     {
20.         int _nrOrd, _startingNode, _destinationNode, _distance, _capacity;
21.
22.         Edge(int nrOrd, int startingNode, int destinationNode, int distance = 0, int
capacity = 0)
23.         {
24.             _nrOrd = nrOrd;
25.             _startingNode = startingNode;
26.             _destinationNode = destinationNode;
27.             _distance = distance;
28.             _capacity = capacity;
29.         }
30.
31.         operator int()
32.         {
33.             return _distance;
34.         }
35.     };
36.
37. private:
38.     bool _oriented;
39.     bool _weighted;
40.     bool _capacity;
41.
42.     int _nrNodes;
43.     int _nrEdges;
44.
45.     vector < vector <Edge> > _neighborList;
46.
47.     //does a bfs on the graph
48.     void BFS(vector<bool>& visitedNodes, vector<int>& distances, int startNode);
49.     //does a dfs on the graph
50.     void DFS(vector<bool>& visitedNodes, int currentNode);
51.     //does a dfs used for biconnected components
52.     void DFS_biconnectedComponents(vector< vector <int> >& bcc, stack<int>& nodeStack,
vector<bool>& visited, vector<int>& depth, vector<int>& lowestDepthReacheable, int
currentNode, int parentNode, int currentDepth);
53.     //tarjan's dfs, used for strongly connected components
54.     void TDFS(vector< vector <int> >& scc, stack<int>& nodeStack, vector<bool>&
stackMember, vector<int>& discoveryOrder, vector<int>& lowestDepthReacheable, int
currentNode, int& currentTime);
55.     //tarjan's dfs, used for critical connections

```

```

56.     void TDFS_criticalConnections(vector< vector<int> >& solution, vector<int>&
       discoveryOrder, vector<int>& lowestDepthReacheable, int currentNode, int parentNode,
       int& currentTime);
57.     //does a dfs, used for the topological sort
58.     void DFS_topologicalSort(stack<int>& result, vector<bool>& visitedNodes, int
       currentNode);
59.     //does dijkstra's algorithm
60.     void Dijkstra(priority_queue < pair<int, int>, vector< pair<int, int> >, greater
       <pair<int, int>> >& heap, vector<bool>& visitedNodes, vector<int>& distances);
61.     //does BellmanFord's algorithm
62.     void BellmanFord(queue<int>& nodeOrder, vector<bool>& inQueue, vector<int>&
       distances, vector<int>& numberOfAparitions, int startNode);
63.     //used in DisjointSet
64.     static int getRootDisjointSet(vector<int>& parent, int currentNode);
65.     //used in DisjointSet
66.     static void changeColorDisjointSet(vector<int>& parent, vector<int>& height, int
       node1, int node2);
67.     //BFS used in Edmonds-Karp's algorithm
68.     bool BFS_maxFlow(vector<vector<int>>& capacity, vector<vector<int>>& flow,
       vector<int>& parent, vector<bool>& visitedNodes, int startNode, int destinationNode);
69.     //Edmonds-Karp's algorithm
70.     int EdmondsKarp(vector<vector<int>>& capacity, vector<vector<int>>& flow,
       vector<int>& parent, vector<bool>& visitedNodes, int startNode, int destinationNode);
71.     //gets an Euler cycle if there is one
72.     void EulerCycle(vector<bool>& visitedEdges, vector<int>& solution, int&
       edgeCounter, int sourceNode);
73.     //cheks if a node is eligible
74.     bool checkNode_HopcroftKarp(vector<int>& leftGraph, vector<int>& rightGraph,
       vector<bool>& visitedNodes, int currentNode);
75.     //returns a vector which is contains the maximum bipartite matching using
       HopcroftKarp's algorithm
76.     void HopcroftKarp(vector<int>& leftGraph, vector<int>& rightGraph);
77.     //reads the Graph since the problem requires an unique reading
78.     void readHamiltonGraph(istream& input);
79.     //determines the cost of the minimal Hamilton cycle, or returns 2e9 if there isn't
       one
80.     int determineMinimalCostHamiltonCycle(vector<vector<int>>& minimalCost);
81.
82.
83. public:
84.     Graph(int nrNodes = 0, int nrEdges = 0, bool oriented = false, bool weighted =
       false, bool capacity = false);
85.     ~Graph() {}
86.
87. #pragma region Setters&Getters
88.     void setOrientation(bool orientation)
89.     {
90.         _oriented = orientation;
91.     }
92.     void setNrNodes(int nrNodes)
93.     {
94.         _nrNodes = nrNodes;
95.     }
96.     void setNrEdges(int nrEdges)
97.     {
98.         _nrEdges = nrEdges;
99.     }
100.
101.     bool getOrientation()
102.     {
103.         return _oriented;

```

```

104.     }
105.     int getNrNodes()
106.     {
107.         return _nrNodes;
108.     }
109.     int getNrEdges()
110.     {
111.         return _nrEdges;
112.     }
113.     #pragma endregion
114.
115.     //reads and builds the graph
116.     void buildNeighborList(istream& input);
117.     //returns a vector of all the edges
118.     vector<Edge> getEdges();
119.     //adds an edge to the graph
120.     void addEdge(Edge newEdge);
121.
122.     //Function Wrappers
123.
124.     //minimal distances from a starting node
125.     vector<int> minimalDistances(int startNode);
126.     //returns the number of conex components
127.     int numberOfConexComponents();
128.     //returns the biconnected components
129.     // de modificat numele
130.     vector< vector <int> > biconnectedComponents();
131.     //returns the strongly connected componentss
132.     //de modificat numele
133.     vector< vector <int> > stronglyConnectedComponents();
134.     //returns the critical connections
135.     vector< vector <int> > criticalConnections();
136.     //does the nodes that exists in all minimal ways between two nodes
137.     vector<int> minimalDistanceBetweenTwoNodes(int startNode, int finishNode);
138.     //returns if a graph exists
139.     bool HavelHakimi(istream& input);
140.     //returns the topological sort
141.     vector<int> topologicalSort();
142.     //returns the minimal positive only weighted distances, using dijkstra's
    algorithm
143.     vector<int> minimalWeightedDistances(int startNode);
144.     //returns minimal negative weighted distances, using BellmanFord's algorithm
145.     vector<int> minimalNegativeWeightedDistances(int startNode);
146.     //the function where getRootDisjointSet and changeColorDisjointSet are used
    in a disjoint set
147.     static void disjointSet(istream& input, ostream& output);
148.     //returns the mst(apm) of the graph
149.     vector<Edge> minimumSpanningTree();
150.     //returns the distance from any node to any node
151.     vector< vector <int> > RoyFloyd();
152.     //returns the diameter of a tree
153.     int diameterOfTree();
154.     //returns the max flow of a network
155.     int maxFlow();
156.     //returns the Euler cycle, if there is one
157.     vector<int> buildEulerCycle();
158.     //returns the maximum bipartite matching
159.     vector<int> maximumBipartiteMatching(int nrNodesRightGraph);
160.     //returns the cost of the minimal Hamilton cycle, or returns 2e9 if there
    isn't one
161.     int minimalCostHamiltonCycle(istream& input);

```

```

162.     };
163.
164.     Graph::Graph(int nrNodes, int nrEdges, bool oriented, bool weighted, bool
capacity)
165.     {
166.         _nrNodes = nrNodes;
167.         _nrEdges = nrEdges;
168.
169.         _oriented = oriented;
170.         _weighted = weighted;
171.         _capacity = capacity;
172.
173.         vector <Edge> emptyVector;
174.
175.         for (int i = 0; i < nrNodes; i++)
176.         {
177.             _neighborList.push_back(emptyVector);
178.         }
179.     }
180.
181.     void Graph::BFS(vector<bool>& visitedNodes, vector<int>& distances, int
startNode)
182.     {
183.         queue<int> nodeOrder;
184.
185.         nodeOrder.push(startNode);
186.         distances[startNode] = 0;
187.
188.         while (nodeOrder.empty() == false)
189.         {
190.             int currentNode = nodeOrder.front();
191.             visitedNodes[currentNode] = true;
192.
193.             for (auto& neighbor : _neighborList[currentNode])
194.             {
195.                 int neighborNode = neighbor._destinationNode;
196.
197.                 if (visitedNodes[neighborNode] == false)
198.                 {
199.                     nodeOrder.push(neighborNode);
200.                     distances[neighborNode] = distances[currentNode] + 1;
201.                     visitedNodes[neighborNode] = true;
202.                 }
203.             }
204.
205.             nodeOrder.pop();
206.         }
207.     }
208.
209.     void Graph::DFS(vector<bool>& visitedNodes, int currentNode)
210.     {
211.         visitedNodes[currentNode] = true;
212.
213.         for (auto& neighbor : _neighborList[currentNode])
214.         {
215.             int neighborNode = neighbor._destinationNode;
216.
217.             if (visitedNodes[neighborNode] == false)
218.             {
219.                 DFS(visitedNodes, neighborNode);
220.             }

```

```

221.     }
222. }
223.
224. void Graph::DFS_biconnectedComponents(vector< vector <int>>& bcc, stack<int>&
    nodeStack, vector<bool>& visited, vector<int>& depth, vector<int>&
    lowestDepthReacheable, int currentNode, int parentNode, int currentDepth)
225. {
226.     depth[currentNode] =
    currentDepth;                                     //initializing the
    depth of the current node
227.     lowestDepthReacheable[currentNode] =
    currentDepth;                                     //initializing the lowest depth
    reacheble time of the current node
228.     visited[currentNode] =
    true;                                             //insert the current
    node in the stack and mark it as visited
229.     nodeStack.push(currentNode);
230.
231.     for (auto& neighbor : _neighborList[currentNode])
232.     {
233.         int neighborNode = neighbor._destinationNode;
234.
235.         if (neighborNode != parentNode)
236.         {
237.             if (visited[neighborNode] == true)
238.             {
239.                 lowestDepthReacheable[currentNode] =
    min(lowestDepthReacheable[currentNode],
    depth[neighborNode]);                             //this is were the back edges are found
240.             }
241.             else
242.             {
243.                 DFS_biconnectedComponents(bcc, nodeStack, visited, depth,
    lowestDepthReacheable, neighborNode, currentNode, currentDepth + 1);
244.
245.                 lowestDepthReacheable[currentNode] =
    min(lowestDepthReacheable[currentNode],
    lowestDepthReacheable[neighborNode]);             //updating with lowest depth that the
    descendants can reach
246.
247.                 if (depth[currentNode] <=
    lowestDepthReacheable[neighborNode])               //if it is true it means that the
    neighbor has no back edge above the current node
248.                 {
249.                     //current node separates two biconnected components
250.                     vector <int> biconnectedComponent;
251.                     biconnectedComponent.push_back(currentNode);
252.
253.                     int node = nodeStack.top();
254.                     nodeStack.pop();
255.                     biconnectedComponent.push_back(node);
256.
257.                     while (node !=
    neighborNode)                                     //pop nodes until we reach
    the neighbor (inclusive) and that is the current biconnected component
258.                     {
259.                         node = nodeStack.top();
260.                         nodeStack.pop();
261.                         biconnectedComponent.push_back(node);
262.                     }

```

```

263.             bcc.push_back(biconnectedComponent);
                //adding it to the rest of the components
264.         }
265.     }
266. }
267. }
268. }
269.
270. void Graph::TDFS(vector< vector <int> >& scc, stack<int>& nodeStack,
vector<bool>& stackMember, vector<int>& discoveryOrder, vector<int>&
lowestDepthReacheable, int currentNode, int& currentTime)
271. {
272.     discoveryOrder[currentNode] =
currentTime; //initializing the discovery time of
the current node
273.     lowestDepthReacheable[currentNode] =
currentTime; //initializing the lowest depth reachble
time of the current node
274.     stackMember[currentNode] =
true; //insert the current node in the
stack and mark it
275.     nodeStack.push(currentNode);
276.     currentTime++;
//increment the time
277.
278.     for (auto& neighbor :
_neighborList[currentNode]) //visit all neibors
279.     {
280.         int neighborNode = neighbor._destinationNode;
281.
282.         if (discoveryOrder[neighborNode] == -
1)
//if it has not been visited yet
283.         {
284.             TDFS(scc, nodeStack, stackMember, discoveryOrder,
lowestDepthReacheable, neighborNode, currentTime); //DFS on
that node
285.
286.             lowestDepthReacheable[currentNode] =
min(lowestDepthReacheable[currentNode],
lowestDepthReacheable[neighborNode]); //actualizing on the recursion
return path
287.         }
288.         else if (stackMember[neighborNode] ==
true) //if the neighbor is on the stack is part of
the current strongly connected component
289.         {
290.             lowestDepthReacheable[currentNode] =
min(lowestDepthReacheable[currentNode], discoveryOrder[neighborNode]);
291.         }
292.     }
293.
294.     if (lowestDepthReacheable[currentNode] ==
discoveryOrder[currentNode]) //if lowestDepthReacheable and discoveryOrder are
equal it means the current node is the root of
295.     {
//the current strongly connected component
296.         vector <int> stronglyConnectedComponent;
297.
298.         int node = nodeStack.top();
299.         nodeStack.pop();

```

```

300.         stackMember[node] = false;
301.         stronglyConnectedComponent.push_back(node);
302.
303.         while (node !=
currentNode) //pop nodes until we reach
current node (inclusive) and that is the current strongly connected component
304.         {
305.             node = nodeStack.top();
306.             nodeStack.pop();
307.             stackMember[node] = false;
308.             stronglyConnectedComponent.push_back(node);
309.         }
310.
311.         scc.push_back(stronglyConnectedComponent);
//adding it to the rest of the components
312.     }
313. }
314.
315. void Graph::TDFS_criticalConnections(vector< vector<int> >& solution,
vector<int>& discoveryOrder, vector<int>& lowestDepthReacheable, int currentNode, int
parentNode, int& currentTime)
316. {
317.     discoveryOrder[currentNode] = currentTime;
318.     lowestDepthReacheable[currentNode] = currentTime;
319.     currentTime++;
320.
321.     for (auto& neighbor : _neighborList[currentNode])
322.     {
323.         int neighborNode = neighbor._destinationNode;
324.
325.         if (discoveryOrder[neighborNode] == -1)
326.         {
327.             TDFS_criticalConnections(solution, discoveryOrder,
lowestDepthReacheable, neighborNode, currentNode, currentTime);
328.
329.             lowestDepthReacheable[currentNode] =
min(lowestDepthReacheable[currentNode], lowestDepthReacheable[neighborNode]);
330.
331.             if (lowestDepthReacheable[neighborNode] >
discoveryOrder[currentNode])
332.             {
333.                 vector <int> result;
334.
335.                 result.push_back(currentNode);
336.                 result.push_back(neighborNode);
337.
338.                 solution.push_back(result);
339.             }
340.
341.         }
342.         else if (neighborNode != parentNode)
343.         {
344.             lowestDepthReacheable[currentNode] =
min(lowestDepthReacheable[currentNode], discoveryOrder[neighborNode]);
345.         }
346.     }
347. }
348.
349. void Graph::DFS_topologicalSort(stack<int>& result, vector<bool>& visitedNodes,
int currentNode)
350. {

```

```

351.         visitedNodes[currentNode] = true;
352.
353.         for (auto& neighbor : _neighborList[currentNode])
354.         {
355.             int neighborNode = neighbor._destinationNode;
356.
357.             if (visitedNodes[neighborNode] == false)
358.             {
359.                 DFS_topologicalSort(result, visitedNodes, neighborNode);
360.             }
361.         }
362.
363.         result.push(currentNode);
364.     }
365.
366.     void Graph::Dijkstra(priority_queue < pair<int, int>, vector< pair<int, int> >,
greater <pair<int, int>>& heap, vector<bool>& visitedNodes, vector<int>& distances)
367.     {
368.         while (heap.empty() ==
false)
            //while the heap has elements
369.         {
370.             pair <int, int> currentPair = heap.top();
371.             heap.pop();
372.
373.             int currentNode = currentPair.second;
374.             int currentDistance = currentPair.first;
375.
376.             visitedNodes[currentNode] = true;
377.
378.             if (currentDistance ==
distances[currentNode])
                //if the current distance is not the same as the one we have in the
distance vector it means that we have
379.             {
                //already found a better way
380.                 for (auto& neighborPair :
_neighborList[currentNode])
                    //visit all neighbors
381.                 {
382.                     int neighborNode = neighborPair._destinationNode;
383.                     int edgeDistance = neighborPair._distance;
384.
385.                     if (visitedNodes[neighborNode] ==
false) //if
the node has been visited we already know the minimal distance to it
386.                     {
387.                         if (distances[neighborNode] > currentDistance +
edgeDistance) //test if we can find a
smaller distance
388.                         {
389.                             distances[neighborNode] = currentDistance +
edgeDistance; //update the minimal
distance if we've found a new one
390.                             heap.push(make_pair(distances[neighborNode],
neighborNode)); //push the node and it's
distance in the heap
391.                         }
392.                     }
393.                 }
394.             }

```



```

395.     }
396. }
397.
398. void Graph::BellmanFord(queue<int>& nodeOrder, vector<bool>& inQueue,
vector<int>& distances, vector<int>& numberOfAparitions, int startNode)
399. {
400.     while (nodeOrder.empty() == false)
401.     {
402.         int currentNode = nodeOrder.front();
403.         nodeOrder.pop();
404.
405.         inQueue[currentNode] = false;
406.         numberOfAparitions[currentNode]++;
407.
408.         if (numberOfAparitions[currentNode] ==
_nrNodes) //if
we've encountered the same node (number of nodes - 1) times it means we are in a
negative loop
409.         {
410.             distances[startNode] = -1;
411.             break;
412.         }
413.
414.         for (auto& neighborPair :
_neighborList[currentNode])
//visit all neighbors
415.         {
416.             int neighborNode = neighborPair._destinationNode;
417.             int edgeDistance = neighborPair._distance;
418.
419.             if (distances[neighborNode] > distances[currentNode] +
edgeDistance) //test if we can find
a smaller distance
420.             {
421.                 distances[neighborNode] = distances[currentNode] +
edgeDistance; //update the minimal
distance if we've found a new one
422.
423.                 if (inQueue[neighborNode] == false)
424.                 {
425.                     nodeOrder.push(neighborNode);
//if it's not in the queue to be
processed, we push it
426.                     inQueue[neighborNode] =
true;
//and check it
427.                 }
428.             }
429.         }
430.     }
431. }
432.
433. int Graph::getRootDisjointSet(vector<int>& parent, int currentNode)
434. {
435.     int root = currentNode;
436.
437.     while (parent[root] != root)
438.     {
439.         root = parent[root];
440.     }
441. }

```

```

442.         while (parent[currentNode] != root) {
443.             int aux = parent[currentNode];
444.             parent[currentNode] = root;
445.             currentNode = aux;
446.         }
447.
448.         return root;
449.     }
450.
451.     void Graph::changeColorDisjointSet(vector<int>& parent, vector<int>& height, int
node1, int node2)
452.     {
453.         int rootNode1 = Graph::getRootDisjointSet(parent, node1);
454.         int rootNode2 = Graph::getRootDisjointSet(parent, node2);
455.
456.         if (rootNode1 == rootNode2)
457.             return;
458.
459.         if (height[rootNode1] < height[rootNode2])
460.         {
461.             parent[rootNode1] = rootNode2;
462.             height[rootNode2] += height[rootNode1];
463.         }
464.         else
465.         {
466.             parent[rootNode2] = rootNode1;
467.             height[rootNode1] += height[rootNode2];
468.         }
469.     }
470.
471.     /// <summary>
472.     /// does a BFS on the network to determine the parents
473.     /// </summary>
474.     /// <param name="capacity"></param>
475.     /// <param name="flow"></param>
476.     /// <param name="parent"></param>
477.     /// <param name="visitedNodes"></param>
478.     /// <param name="startNode"></param>
479.     /// <param name="destinationNode"></param>
480.     /// <returns></returns>
481.     bool Graph::BFS_maxFlow(vector<vector<int>>& capacity, vector<vector<int>>&
flow, vector<int>& parent, vector<bool>& visitedNodes, int startNode, int
destinationNode)
482.     {
483.         queue<int> nodeOrder;
484.         nodeOrder.push(startNode);
485.
486.         for (int i = 0; i < _nrNodes; i++)
487.         {
488.             visitedNodes[i] = false;
489.         }
490.
491.         while (nodeOrder.empty() == false)
492.         {
493.             int node = nodeOrder.front();
494.             nodeOrder.pop();
495.             visitedNodes[node] = true;
496.
497.             if (node != destinationNode)
498.             {
499.                 for (auto& edge : _neighborList[node])

```

```

500.         {
501.             if (capacity[node][edge._destinationNode] ==
502.                 flow[node][edge._destinationNode] || visitedNodes[edge._destinationNode] == true)
503.             {
504.                 continue;
505.             }
506.             parent[edge._destinationNode] = node;
507.             nodeOrder.push(edge._destinationNode);
508.         }
509.     }
510. }
511.
512.     return visitedNodes[destinationNode];
513. }
514.
515. int Graph::EdmondsKarp(vector<vector<int>>& capacity, vector<vector<int>>& flow,
516.     vector<int>& parent, vector<bool>& visitedNodes, int startNode, int destinationNode)
517. {
518.     int max_flow = 0;
519.     while (BFS_maxFlow(capacity, flow, parent, visitedNodes, startNode,
520.         destinationNode) == true) //while there is a way
521.     {
522.         for (auto& edge : _neighborList[destinationNode])
523.         {
524.             int node = edge._destinationNode;
525.             if (flow[node][destinationNode] == capacity[node][destinationNode]
526.                 || visitedNodes[node] == false)
527.             {
528.                 continue; //if the flow is already at the
529.                 //maximum or if the node is not part of our way we skip
530.             }
531.             int minflow = 2e9;
532.             parent[destinationNode] =
533.                 node; //initialize the parent
534.             for (int vertex = destinationNode; vertex != startNode; vertex =
535.                 parent[vertex])
536.             {
537.                 minflow = min(minflow, capacity[parent[vertex]][vertex] -
538.                     flow[parent[vertex]][vertex]); //determine the minflow of a way
539.             }
540.             if (minflow != 0)
541.             {
542.                 for (int vertex = destinationNode; vertex != startNode; vertex =
543.                     parent[vertex])
544.                 {
545.                     flow[parent[vertex]][vertex] +=
546.                         minflow; //add the minflow
547.                     flow[vertex][parent[vertex]] -=
548.                         minflow; //subtract the
549.                         reverse way
550.                 }
551.             }
552.         }
553.     }
554. }

```

```

547.             max_flow +=
minflow;
        //add to the overall flow
548.    }
549.    }
550.    }
551.
552.    return max_flow;
553.    }
554.
555.    void Graph::EulerCycle(vector<bool>& visitedEdges, vector<int>& solution, int&
edgeCounter, int sourceNode)
556.    {
557.        while (_neighborList[sourceNode].empty() == false)
558.        {
559.            Edge edge = _neighborList[sourceNode].back();
560.
561.            int destinationNode = edge._destinationNode;
562.            int edgeNrOrd = edge._nrOrd;
563.
564.            _neighborList[sourceNode].pop_back();
565.
566.            if (visitedEdges[edgeNrOrd] == false)
567.            {
568.                visitedEdges[edgeNrOrd] = true;
569.
570.                EulerCycle(visitedEdges, solution, edgeCounter, destinationNode);
571.            }
572.        }
573.
574.        solution.push_back(sourceNode);
575.        edgeCounter++;
576.    }
577.
578.    bool Graph::checkNode_HopcroftKarp(vector<int>& leftGraph, vector<int>&
rightGraph, vector<bool>& visitedNodes, int currentNode)
579.    {
580.        if (visitedNodes[currentNode] == true) //return
if the node is already visited
581.        {
582.            return false;
583.        }
584.
585.        visitedNodes[currentNode] = true;
586.
587.        for (auto& edge : _neighborList[currentNode])
588.        {
589.            int destinationNode = edge._destinationNode;
590.
591.            if (rightGraph[destinationNode] == -1) //if we
find a node without match
592.            {
593.                leftGraph[currentNode] = destinationNode; //we
match them
594.                rightGraph[destinationNode] = currentNode; //we
match them
595.
596.                return true;
597.            }
598.        }
599.

```

```

600.         for (auto& edge : _neighborList[currentNode])
601.         {
602.             int destinationNode = edge._destinationNode;
603.
604.             if (checkNode_HopcroftKarp(leftGraph, rightGraph, visitedNodes,
rightGraph[destinationNode]) == true)
605.             {
606.                 leftGraph[currentNode] = destinationNode;
607.                 rightGraph[destinationNode] = currentNode;
608.
609.                 return true;
610.             }
611.         }
612.
613.         return false;
614.     }
615.
616.     void Graph::HopcroftKarp(vector<int>& leftGraph, vector<int>& rightGraph)
617.     {
618.         bool change = true;
619.
620.         while (change == true)
621.         {
622.             change = false;
623.
624.             int size = max(leftGraph.size(), rightGraph.size());
625.             vector<bool> visitedNodes(size, false);
626.
627.             for (int i = 0; i < _nrNodes; i++)
628.             {
629.                 if (leftGraph[i] == -
1) //check if a
node is already matched
630.                 {
631.                     if (checkNode_HopcroftKarp(leftGraph, rightGraph, visitedNodes,
i) == true)
632.                     {
633.                         change = true;
634.                     }
635.                 }
636.             }
637.         }
638.     }
639.
640.     void Graph::readHamiltonGraph(istream& input)
641.     {
642.         int node1, node2, cost;
643.
644.         for (int i = 0; i < _nrEdges; i++)
645.         {
646.             input >> node1 >> node2 >> cost;
647.
648.             Edge newEdge(i, node2, node1, cost, 0);
649.
650.             _neighborList[node2].push_back(newEdge);
651.         }
652.     }
653.
654.     int Graph::determineMinimalCostHamiltonCycle(vector<vector<int>>& minimalCost)
655.     {

```

```

656.         for (int i = 0; i < (1 << _nrNodes); i++) //every
        combination of nodes (if the current bit is set to 1, it is part of the chain)
657.         {
658.             for (int j = 0; j < _nrNodes; j++) //the node we
        wish to add to the current chain
659.             {
660.                 if ( (i & (1 << j)) != 0) //check if the
        node j is in the current chain
661.                 {
662.                     for (auto& edge : _neighborList[j])
663.                     {
664.                         int node = edge._destinationNode;
665.                         int cost = edge._distance;
666.
667.                         if ( (i & (1 << node)) != 0) //check if the
        node is in the current chain
668.                         {
669.                             minimalCost[i][j] = min(minimalCost[i][j], minimalCost[i
        ^ (1 << j)][node] + cost);
670.                             //update the minimal cost using i nodes that end in j
671.                         }
672.                     }
673.                 }
674.             }
675.         }
676.
677.         int minCost =
        2e9; //initialize
        the minimal cost
678.         for (auto& edge : _neighborList[0])
679.         {
680.             int node = edge._destinationNode;
681.             int cost = edge._distance;
682.
683.             minCost = min(minCost, minimalCost[((1 << _nrNodes) - 1)][node] +
        cost); //determine the minimal cost
684.         }
685.
686.         return minCost;
687.     }
688.
689.
690.
691.
692.     void Graph::buildNeighborList(istream& input)
693.     {
694.         int node1, node2, distance, capacity;
695.
696.         for (int i = 0; i < _nrEdges; i++)
697.         {
698.             input >> node1 >> node2;
699.
700.             node1--;
701.             node2--;
702.
703.             if (_weighted == true)
704.             {
705.                 fin >> distance;
706.             }
707.             else
708.             {

```

```

709.         distance = 0;
710.     }
711.
712.     if (_capacity == true)
713.     {
714.         fin >> capacity;
715.     }
716.     else
717.     {
718.         capacity = 0;
719.     }
720.
721.     Edge newEdge(i, node1, node2, distance, capacity);
722.
723.     _neighborList[node1].push_back(newEdge);
724.
725.     if (_oriented == false)
726.     {
727.         Edge newEdge(i, node2, node1, distance, capacity);
728.
729.         _neighborList[node2].push_back(newEdge);
730.     }
731. }
732. }
733.
734. vector<Graph::Edge> Graph::getEdges()
735. {
736.     vector<Edge> edges;
737.
738.     if (_oriented == true)
739.     {
740.         for (int node = 0; node < _nrNodes; node++)
741.         {
742.             for (auto& edge : _neighborList[node])
743.             {
744.                 edges.push_back(edge);
745.             }
746.         }
747.     }
748.     else
749.     {
750.         vector<bool> visited(_nrNodes, false);
751.
752.         for (int node = 0; node < _nrNodes; node++)
753.         {
754.             for (auto& edge : _neighborList[node])
755.             {
756.                 if (visited[edge._destinationNode] == false)
757.                 {
758.                     edges.push_back(edge);
759.                 }
760.             }
761.
762.             visited[node] = true;
763.         }
764.     }
765.
766.     return edges;
767. }
768.
769. void Graph::addEdge(Edge newEdge)

```

```

770.     {
771.         _neighborList[newEdge._startingNode].push_back(newEdge);
772.
773.         if (_oriented == false)
774.         {
775.             Edge newEdge2(newEdge._destinationNode, newEdge._startingNode,
newEdge._distance, newEdge._capacity);
776.             _neighborList[newEdge._destinationNode].push_back(newEdge2);
777.         }
778.     }
779.
780.     _nrEdges++;
781. }
782.
783. vector<int> Graph::minimalDistances(int startNode)
784. {
785.     vector <bool> visited(_nrNodes, false);
786.     vector <int> distances(_nrNodes, -1);
787.
788.     BFS(visited, distances, startNode);
789.
790.     return distances;
791. }
792.
793. int Graph::numberOfConexComponents()
794. {
795.     int numberOfConexComponents = 0;
796.
797.     vector <bool> visited(_nrNodes, false);
798.
799.     for (int i = 0; i < _nrNodes; i++)
800.     {
801.         if (visited[i] == false)
802.         {
803.             DFS(visited, i);
804.             numberOfConexComponents++;
805.         }
806.     }
807.
808.     return numberOfConexComponents;
809. }
810.
811. vector< vector <int> >
Graph::biconnectedComponents()           //initializing all data
structures needed for the Biconnected DFS Algorithm
812. {
813.     vector< vector<int> >
bcc;                                     //this will contain all
Biconnected Components
814.
815.     stack<int>
nodeStack;                             //will store the
connected ancestors
816.
817.     vector<bool> visited(_nrNodes,
false);                                //will check whether a node has been
visited or not
818.     vector<int> depth(_nrNodes, -
1);                                    //will cotain the depth of every
node

```



```

819.         vector<int> lowestDepthReacheable(_nrNodes, -
1);                                     //will contain the lowest depth a node can reach
        through its descendants
820.
821.         DFS_biconnectedComponents(bcc, nodeStack, visited, depth,
lowestDepthReacheable, 0, -1, 0);
822.
823.         return bcc;
824.     }
825.
826.     vector< vector<int> >
Graph::stronglyConnectedComponents()           //initializing all data
        structures needed for the Tarjan's DFS Algorithm
827.     {
828.         vector< vector<int> >
scc;                                           //this will contain all
        Strongly Connected Components
829.
830.         stack<int>
nodeStack;                                   //will store the
        connected ancestors
831.
832.         vector<bool> stackMember(_nrNodes,
false);                                     //will check whether a node is in stack
833.         vector<int> discoveryOrder(_nrNodes, -
1);                                         //contains the order in which the nodes are
        discovered
834.         vector<int> lowestDepthReacheable(_nrNodes, -
1);                                         //the lowest ancestor that can be reached
835.
836.         int currentTime =
0;                                           //a timer used for
        discoveryOrder and lowestDepthReacheable
837.
838.         for (int i = 0; i < _nrNodes; i++)
839.         {
840.             if (discoveryOrder[i] == -
1)                                         //if the node hasn't been discovered
        yet, we start the DFS from him
841.             {
842.                 TDFS(scc, nodeStack, stackMember, discoveryOrder,
lowestDepthReacheable, i, currentTime);     //the tarjan's algorithm
843.             }
844.         }
845.
846.         return scc;
847.     }
848.
849.     vector<vector<int>> Graph::criticalConnections()
850.     {
851.         vector< vector<int> > solution;
852.
853.         vector<int> discoveryOrder(_nrNodes, -1);
854.         vector<int> lowestDepthReacheable(_nrNodes, -1);
855.
856.         int currentTime = 0;
857.
858.         for (int i = 0; i < _nrNodes; i++)
859.         {
860.             if (discoveryOrder[i] == -1)
861.             {

```

```

862.         TDFS_criticalConnections(solution, discoveryOrder,
lowestDepthReacheable, i, -1, currentTime);
863.     }
864. }
865.
866.     return solution;
867. }
868.
869.     vector<int> Graph::minimalDistanceBetweenTwoNodes(int startNode, int finishNode)
870.     {
871.         vector<int> normalWay(_nrNodes, -
1); //will contain the values from the bfs
starting from the starting node
872.         vector<int> reverseWay(_nrNodes, -
1); //will contain the values from the bfs
starting from the finishing node
873.         vector<int> frequency(_nrNodes,
0); //will contain the frequency of the nodes, if a
node is on all of the minimal ways between the starting node and the finishing node,
874.         vector<int>
result; //then his distance
will be the same from those nodes
875.
876.         normalWay =
minimalDistances(startNode); //getting the minimal
distances in the normal way
877.         reverseWay =
minimalDistances(finishNode); //getting the minimal
distances in the reverse way
878.
879.         int minimWay =
normalWay[finishNode]; //the minimal distance
between the starting node and the finishing node
880.
881.         for (int i = 0; i < _nrNodes; i++)
882.             if (normalWay[i] + reverseWay[i] == minimWay)
883.                 frequency[normalWay[i]]++;
884.
885.         for (int i = 0; i < _nrNodes; i++)
886.             if (normalWay[i] + reverseWay[i] == minimWay && frequency[normalWay[i]]
== 1)
887.                 result.push_back(i);
888.
889.         return result;
890.     }
891.
892.     bool Graph::HavelHakimi(istream& input)
893.     {
894.         vector<int> degrees;
895.         int totalDegree = 0;
896.
897.         for (int i = 0; i < _nrNodes; i++)
898.         {
899.             int degree;
900.             fin >> degree;
901.
902.             if (degree < 0 || degree > _nrNodes - 1)
903.             {
904.                 return false;
905.             }
906.

```

```

907.         degrees.push_back(degree);
908.         totalDegree += degree;
909.     }
910.
911.     sort(degrees.begin(), degrees.end(), greater<int>());
912.
913.     if (totalDegree % 2 == 1 || _nrNodes == 1)
914.     {
915.         return false;
916.     }
917.
918.     while (degrees[1] != 0)
919.     {
920.         for (auto degree : degrees)
921.             fout << degree << " ";
922.         fout << "\n";
923.
924.         for (int i = 1; i <= degrees[0]; i++)
925.         {
926.             if (degrees[i] != 0)
927.             {
928.                 degrees[i]--;
929.             }
930.             else
931.             {
932.                 return false;
933.             }
934.         }
935.
936.         degrees[0] = 0;
937.
938.         sort(degrees.begin(), degrees.end(), greater<int>());
939.     }
940.
941.     if (degrees[0] == 0)
942.     {
943.         return true;
944.     }
945.     else
946.     {
947.         return false;
948.     }
949. }
950.
951. vector<int> Graph::topologicalSort()
952. {
953.     vector<bool> visited(_nrNodes + 1, false);
954.     vector<int> solution;
955.
956.     stack<int> result;
957.
958.     for (int i = 0; i < _nrNodes; i++)
959.     {
960.         if (visited[i] == false)
961.         {
962.             DFS_topologicalSort(result, visited, i);
963.         }
964.     }
965.
966.     while (result.empty() == 0)
967.     {

```

```

968.         solution.push_back(result.top());
969.         result.pop();
970.     }
971.
972.     return solution;
973. }
974.
975.     /// <summary>
976.     /// complexity O(N + MlogN), using priority queue
977.     /// </summary>
978.     /// <param name="startNode"></param>
979.     /// <returns></returns>
980.     vector<int> Graph::minimalWeightedDistances(int startNode)
981.     {
982.         priority_queue < pair<int, int>, vector< pair<int, int> >, greater<pair<int,
int>> > heap;
983.
984.         vector <bool> visited(_nrNodes, false);
985.         vector <int> distances(_nrNodes, 2e9);
986.
987.         distances[startNode] = 0;
988.
989.         heap.push(make_pair(0, startNode));
990.
991.         Dijkstra(heap, visited, distances);
992.
993.         return distances;
994.     }
995.
996.     /// <summary>
997.     /// complexity O(n * m), using queue
998.     /// </summary>
999.     /// <param name="startNode"></param>
1000.    /// <returns></returns>
1001.    vector<int> Graph::minimalNegativeWeightedDistances(int startNode)
1002.    {
1003.        queue<int> nodeOrder;
1004.
1005.        vector <bool> inQueue(_nrNodes, false);
1006.        vector <int> distances(_nrNodes, 2e9);
1007.        vector <int> numberOfAparitions(_nrNodes, 0);
1008.
1009.        distances[startNode] =
            0;
1010.        //initialize the distance from the starting node to himself
        nodeOrder.push(startNode);
1011.        //push it in the queue
        inQueue[startNode] =
            true;
1012.        //and check it
1013.        BellmanFord(nodeOrder, inQueue, distances, numberOfAparitions, startNode);
1014.
1015.        return distances;
1016.    }
1017.
1018.    void Graph::disjointSet(istream& in, ostream& out)
1019.    {
1020.        int nrNodes, nrRequests;
1021.
1022.        in >> nrNodes >> nrRequests;

```

```

1023.
1024.     vector<int> parent(nrNodes);
1025.     vector<int> height(nrNodes, 1);
1026.
1027.     for (int i = 0; i < nrNodes; i++)
1028.         parent[i] = i;
1029.
1030.     for (int i = 0; i < nrRequests; i++)
1031.     {
1032.         int node1, node2, request;
1033.
1034.         in >> request >> node1 >> node2;
1035.
1036.         node1--;
1037.         node2--;
1038.
1039.         if (request == 1)
1040.         {
1041.             Graph::changeColorDisjointSet(parent, height, node1, node2);
1042.         }
1043.         else
1044.         {
1045.             if (Graph::getRootDisjointSet(parent, node1) ==
1046.                 Graph::getRootDisjointSet(parent, node2))
1047.             {
1048.                 out << "DA\n";
1049.             }
1050.             else
1051.             {
1052.                 out << "NU\n";
1053.             }
1054.         }
1055.     }
1056.
1057.     vector<Graph::Edge> Graph::minimumSpanningTree()
1058.     {
1059.         vector<Edge> mst;
1060.         vector<Edge> sortedEdges = getEdges();
1061.         vector<int> parent(_nrNodes);
1062.         vector<int> height(_nrNodes, 1);
1063.
1064.         for (int i = 0; i < _nrNodes; i++)
1065.             parent[i] = i;
1066.
1067.         sort(sortedEdges.begin(), sortedEdges.end());
1068.
1069.         int i = 0, nrEdges = 0;
1070.
1071.         while (i < _nrEdges && nrEdges < _nrEdges - 1)
1072.         {
1073.
1074.             if (getRootDisjointSet(parent, sortedEdges[i]._startingNode) !=
1075.                 getRootDisjointSet(parent, sortedEdges[i]._destinationNode))
1076.             {
1077.                 changeColorDisjointSet(parent, height, sortedEdges[i]._startingNode,
1078.                     sortedEdges[i]._destinationNode);
1079.                 mst.push_back(sortedEdges[i]);
1080.                 nrEdges++;
1081.             }

```

```

1081.
1082.         i++;
1083.     }
1084.
1085.     return mst;
1086. }
1087.
1088.     /// <summary>
1089.     /// complexity O(n^3)
1090.     /// </summary>
1091.     /// <returns></returns>
1092.     vector<vector<int>> Graph::RoyFloyd()
1093.     {
1094.         vector<vector<int>> minimalDistances(_nrNodes, vector<int>(_nrNodes, 2e8));
1095.
1096.         for (int i = 0; i < _nrNodes; i++)
1097.         {
1098.             int j = 0;
1099.
1100.             for (auto edge : _neighborList[i])
1101.             {
1102.                 if (edge._distance != 0)
1103.                 {
1104.                     minimalDistances[i][j] = edge._distance;
1105.                 }
1106.
1107.                 j++;
1108.             }
1109.         }
1110.
1111.         for (int k = 0; k < _nrNodes; k++)
1112.         {
1113.             for (int i = 0; i < _nrNodes; i++)
1114.             {
1115.                 for (int j = 0; j < _nrNodes; j++)
1116.                 {
1117.                     if (minimalDistances[i][j] > minimalDistances[i][k] +
minimalDistances[k][j])
1118.                     {
1119.                         minimalDistances[i][j] = minimalDistances[i][k] +
minimalDistances[k][j];
1120.                     }
1121.                 }
1122.             }
1123.         }
1124.
1125.         for (int i = 0; i < _nrNodes; i++)
1126.         {
1127.             for (int j = 0; j < _nrNodes; j++)
1128.             {
1129.                 if (minimalDistances[i][j] == 2e8 || i == j)
1130.                 {
1131.                     minimalDistances[i][j] = 0;
1132.                 }
1133.             }
1134.         }
1135.
1136.         return minimalDistances;
1137.     }
1138.
1139.     int Graph::diameterOfTree() {

```

```

1140.
1141.     vector<int> distances = minimalDistances(0);
1142.
1143.     int maxIndex = 0, maximum = distances[0];
1144.
1145.     for (int i = 0; i < _nrNodes; i++)
1146.     {
1147.         if (distances[i] > maximum)
1148.         {
1149.             maximum = distances[i];
1150.             maxIndex = i;
1151.         }
1152.     }
1153.
1154.     distances = minimalDistances(maxIndex);
1155.
1156.     for (int i = 0; i < _nrNodes; i++)
1157.     {
1158.         if (distances[i] > maximum)
1159.         {
1160.             maximum = distances[i];
1161.         }
1162.     }
1163.
1164.     return maximum + 1;
1165. }
1166.
1167.     /// <summary>
1168.     /// will determine the max flow of a network, using Edmonds Krap's algorithm
1169.     /// complexity  $O((N + M) * N * M)$ 
1170.     /// </summary>
1171.     /// <returns></returns>
1172.     int Graph::maxFlow()
1173.     {
1174.         vector<vector<int>> capacity(_nrNodes, vector<int>(_nrNodes,
1175. //matrix of capacities
1176.         vector<vector<int>> flow(_nrNodes, vector<int>(_nrNodes,
1177. //matrix of flows
1178.         vector<int> parent(_nrNodes, -
1179. //vector of
1180.         parents
1181.         vector<bool> visitedNodes(_nrNodes,
1182. //vector of the
1183.         false);
1184.         visited nodes
1185.
1186.         int maxFlow = 0;
1187.
1188.         for (auto& edge :
1189.         getEdges())
1190.         /
1191.         /add the capacity of all the edges to the matrix of capacities
1192.         {
1193.             capacity[edge._startingNode][edge._destinationNode] = edge._capacity;
1194.         }
1195.
1196.         for (auto& edge : getEdges())
1197.         {
1198.             Edge newEdge(edge._destinationNode, edge._startingNode, edge._distance,
1199.             edge._capacity);
1200.             //add the reverse way
1201.
1202.             addEdge(newEdge);
1203.         }

```

```

1192.
1193.         maxFlow = EdmondsKarp(capacity, flow, parent, visitedNodes, 0, _nrNodes -
1194.         1);                                //obtains the max flow of the network
1195.         return maxFlow;
1196.     }
1197.
1198.     vector<int> Graph::buildEulerCycle()
1199.     {
1200.         vector<bool> visitedEdges(_nrEdges, false);
1201.         vector<int> solution;
1202.         int edgeCounter = 0;
1203.
1204.         for (int i = 0; i < _nrNodes; i++)
1205.         {
1206.             if (_neighborList[i].size() % 2 == 1)
1207.             {
1208.                 solution.push_back(-1);
1209.                 return solution;
1210.             }
1211.         }
1212.
1213.         EulerCycle(visitedEdges, solution, edgeCounter, 0);
1214.
1215.         if (edgeCounter != _nrEdges + 1)
1216.         {
1217.             solution[0] = -1;
1218.             return solution;
1219.         }
1220.         else
1221.         {
1222.             return solution;
1223.         }
1224.     }
1225.
1226.     vector<int> Graph::maximumBipartiteMatching(int nrNodesRightGraph)
1227.     {
1228.         vector<int> leftGraph(_nrNodes, -1);                //initialize
1229.         left subgraph
1230.         vector<int> rightGraph(nrNodesRightGraph, -1);      //initialize
1231.         right subgraph
1232.         HopcroftKarp(leftGraph, rightGraph);
1233.         return leftGraph;
1234.     }
1235.
1236.     int Graph::minimalCostHamiltonCycle(istream& input)
1237.     {
1238.         vector<vector<int>> minimalCost((1 << _nrNodes), vector<int>(_nrNodes,
1239.         2e9));
1240.         //minimalCost[i][j] contains the minimal cost that end in j and has all the
1241.         nodes that are set in i
1242.         minimalCost[1][0] = 0;
1243.         int minCost;
1244.         readHamiltonGraph(input);
1245.         //read the graph

```



```
1246.         minCost =
            determineMinimalCostHamiltonCycle(minimalCost);           //determine the
            minimal cost
1247.
1248.         return minCost;
1249.     }
1250.
1251.
1252.
1253.
1254.     int main()
1255.     {
1256.         //cod problema
1257.
1258.         fin.close();
1259.         fout.close();
1260.
1261.         return 0;
1262.     }
```